

COMP24112_CW_c43594sg

Sarah Guessoum

May 2026

COMP24112 Coursework Report

1. Linear Regression

In Section 1, a linear regression model was implemented using gradient descent together with the Huber loss function. The Huber loss combines the advantages of mean squared error and mean absolute error by behaving quadratically for small residuals and linearly for large residuals, making optimisation less sensitive to outliers while remaining differentiable.

For the linear model, predictions are computed using:

$$\hat{y} = Xw$$

where X is the feature matrix and w is the weight vector. The residual is defined as:

$$r = \hat{y} - y$$

For small residual values, the Huber gradient behaves similarly to squared loss:

$$\frac{\partial L}{\partial r} = r$$

while for larger residuals it becomes:

$$\frac{\partial L}{\partial r} = \delta \cdot \text{sign}(r)$$

Using the chain rule, the gradient with respect to the weights is:

$$\nabla_w L = X^T g(r)$$

where $g(r)$ represents the piecewise derivative of the Huber loss. The model parameters were updated iteratively using gradient descent:

$$w := w - \eta \nabla_w L$$

where η is the learning rate.

One important lesson learned during implementation was the importance of feature standardisation when using gradient based optimisation. Standardising the features improved optimisation stability and accelerated convergence because all features contributed more evenly during weight updates. Another key lesson was the importance of monitoring both the objective function and RMSE during training. The objective function measures optimisation progress, while RMSE provides a more interpretable measure of prediction error on both training and testing sets.

The plots produced in Section 1.2 showed that both the training objective and RMSE decreased rapidly during early iterations before gradually stabilising, indicating successful convergence of the optimisation process. The testing RMSE remained reasonably close to the training RMSE, suggesting that the model generalised relatively well to unseen data and did not severely overfit the training set. However, some prediction error remained because the Diabetes dataset likely contains more complex non linear relationships that cannot be fully captured by a simple linear model.

The learning rate experiments in Section 1.3 demonstrated that hyperparameter selection significantly affects model behaviour. Very small learning rates caused slow convergence because parameter updates were too small, requiring many iterations to minimise the objective function. Larger learning rates accelerated convergence, but excessively large values risked unstable optimisation because updates may overshoot the optimal solution. Intermediate learning rates therefore achieved the best balance between convergence speed, optimisation stability, and testing RMSE performance.

2. Neural Network Classification

The model selection experiments in Section 2.2 demonstrated that neural network architecture and regularisation strongly influence classification performance. Larger hidden layer configurations generally achieved higher classification accuracy and macro-F1 scores because they were capable of learning more complex non linear decision boundaries from the FashionMNIST dataset. However, increasing model complexity also increased computational cost and training time, demonstrating the trade off between predictive performance and efficiency.

The regularisation parameter alpha influenced the balance between fitting the training data and generalising to unseen examples. Smaller alpha values reduced regularisation strength and improved fitting performance, while larger values helped reduce overfitting by penalising excessively large weights. Macro F1 was particularly important because it evaluates performance across all classes rather than relying only on overall accuracy. Some configurations produced convergence warnings because the optimiser reached the maximum iteration limit before fully converging, suggesting that additional iterations may further improve performance. Nevertheless, the selected MLP achieved a strong balance between accuracy, macro F1, efficiency, and model complexity.

In Section 2.3, a convolutional neural network (CNN) was implemented for image classification because convolutional architectures are better suited to preserving spatial relationships within image data. The CNN consisted of convolutional layers with ReLU activations and max pooling layers followed by fully connected layers with dropout regularisation. The convolutional layers extracted local image features such as edges, textures, and shapes, while pooling reduced spatial dimensions and computational complexity. Dropout was included to reduce overfitting by preventing excessive reliance on specific neurons during training.

The model was trained using the Adam optimiser together with CrossEntropyLoss, which is appropriate for multi class classification problems. The training curves showed decreasing loss and increasing accuracy over multiple epochs, indicating stable optimisation and successful feature learning. When evaluated on the same held-out train/test split used for the MLP experiments, the CNN achieved stronger classification performance because convolutional architectures are more effective at capturing spatial image structure.

In Section 2.4, external robustness testing was conducted by contaminating the FashionMNIST test images with Gaussian noise and missing value style degradation. This simulated more realistic real world scenarios where input data may be noisy, incomplete, or partially corrupted. Although classification accuracy and macro F1 decreased slightly compared with the clean test set, the model still maintained relatively strong performance, suggesting that it learned robust and generalisable image features rather than memorising the training data.

Several methods contributed to robustness. Input normalisation improved optimisation stability and ensured consistent numerical scaling of pixel values. Dropout regularisation encouraged the network to rely on distributed feature representations rather than memorising individual examples. In addition, the convolutional architecture itself improved robustness because local feature extraction allows CNNs to remain effective even when parts of the image are partially corrupted. These methods helped the saved scikit-learn model remain more reliable when tested on corrupted external inputs.